Antigravity Master Instruction: Build a Password Manager App

You are building a password manager app similar to 1Password using Antigravity.

You must use the uploaded database schema file, field definitions, and UI screenshots as the source of truth.

Do not invent table names, field names, UI layouts, or major features unless required to make the app work. When something is missing, clearly explain the gap and propose the safest minimal solution.

Security is the highest priority.

Never store plaintext passwords, secure notes, secrets, vault data, recovery keys, or encryption keys in the database.

Use client-side encryption for sensitive vault data before saving it.

Global Rules for Every Step

Before coding each step:

1. Inspect the existing codebase.
2. Inspect the uploaded schema.
3. Inspect the uploaded UI screenshots.
4. Identify which files need changes.
5. Explain the implementation approach briefly.
6. Then implement.

After coding each step:

1. List files created or modified.
2. Explain what was completed.
3. Explain any assumptions.
4. Run available checks/tests/build.
5. Fix errors before moving to the next step.

Do not skip security checks.

Prompt 1: Analyze Schema, Screenshots, and App Requirements

Analyze the uploaded database schema and UI screenshots.

Produce:

1. Database table summary
2. Field-by-field explanation
3. Entity relationship summary
4. Authentication model
5. Vault/item/note/category relationship model
6. UI screen inventory
7. Required app routes
8. Required components
9. Missing fields or unclear schema issues
10. Security risks in the current schema
11. Recommended implementation order

Do not write code yet.

Prompt 2: Create App Architecture

Create the base architecture for the app.

Implement:

- Project folder structure
- Routing
- Auth layout
- Dashboard layout
- Protected routes
- Shared UI components
- Global styles
- Environment variable setup
- Database client setup
- Type definitions from the schema
- Error/loading/empty state components

Follow the uploaded UI screenshots.

After implementation, run the app and fix startup/build errors.

Prompt 3: Database Layer

Create a clean database service layer using the exact uploaded schema.

Implement typed functions for:

- Users/profile
- Vaults
- Password items
- Secure notes
- Categories/folders
- Favorites
- Settings
- Any other schema tables

Requirements:

- Use exact table and column names.
- Add TypeScript types/interfaces.
- Add CRUD functions.
- Add user-level filtering.
- Add safe error handling.
- Do not expose sensitive fields unnecessarily.

Also verify whether row-level security or access policies are needed. If missing, propose and implement safe policies if supported.

Prompt 4: Authentication

Implement authentication.

Required flows:

- Sign up
- Sign in
- Sign out
- Session persistence
- Protected dashboard routes
- Redirect unauthenticated users to login
- Redirect authenticated users to dashboard
- Account/profile page

Security requirements:

- Never store plaintext master passwords.

- Never log passwords or secrets.
- Validate all inputs.
- Use safe error messages.
- Ensure users can only access their own data.

Match the uploaded UI screenshots.

Prompt 5: Client-Side Encryption Foundation

Implement the encryption layer before building vault CRUD.

Requirements:

- Sensitive password-manager data must be encrypted before database storage.
- Use trusted browser cryptography APIs or vetted libraries.
- Do not invent custom cryptography.
- Create reusable encrypt/decrypt utilities.
- Encrypt password values, secure note content, and other secret fields.
- Keep searchable non-sensitive metadata separate where necessary.
- Do not store encryption keys in plaintext.
- Document encryption flow in code comments and README.

Explain:

- What is encrypted
- What is not encrypted
- How keys are derived
- Where keys live during a session
- Current MVP limitations
- What must be improved before production

Prompt 6: Main Dashboard UI

Build the main password manager dashboard according to the screenshots.

Include:

- Sidebar
- Vault/category navigation
- Item list
- Item detail panel
- Search bar
- Add item button

- Favorites section
- Empty states
- Loading states
- Responsive desktop/tablet/mobile behavior

Use the screenshot design closely.

Prompt 7: Password Item CRUD

Implement full password item functionality using the schema.

Features:

- Create password item
- View password item
- Edit password item
- Delete with confirmation
- Hide/show password
- Copy username
- Copy password
- Open website URL
- Mark/unmark favorite
- Assign to vault/category/folder
- Track created/updated dates if schema supports it

Validation:

- Required title/name
- Valid website URL when provided
- Safe password handling
- No empty save
- No unauthorized access

Sensitive fields must be encrypted before saving.

Prompt 8: Secure Notes

Implement secure notes.

Features:

- Create note
- View note
- Edit note

- Delete note
- Search note metadata
- Assign to vault/category/folder if supported
- Favorite note if supported

Note content must be encrypted before saving.

Prompt 9: Vaults, Categories, and Folders

Implement vault/category/folder management using the uploaded schema.

Features:

- Create
- Rename
- Delete with confirmation
- Filter items by vault/category/folder
- Move items between vaults/folders
- Show item counts
- Handle empty states

Rules:

- Users only see their own data.
- Deleting a container must not accidentally expose or orphan sensitive data.
- If cascade behavior is unclear, ask internally through code comments and implement the safest option.

Prompt 10: Search, Filter, and Sort

Implement search and filtering.

Search across safe metadata such as:

- Item title
- Username if stored as searchable metadata
- Website URL
- Category
- Vault

Do not decrypt every secret unnecessarily.

Add filters:

- Favorites
- Vault
- Category/folder
- Recently updated
- Weak passwords if implemented

Add sorting:

- Name
- Created date
- Updated date
- Website

Use debounce for search.

Prompt 11: Password Generator

Implement password generator.

Options:

- Length
- Uppercase
- Lowercase
- Numbers
- Symbols
- Exclude ambiguous characters

Features:

- Generate password
- Copy generated password
- Insert into create/edit item form
- Show password strength

Do not save generated passwords unless the user saves the item.

Prompt 12: Security Health Page

Create a basic security health page.

Include:

- Weak passwords
- Reused passwords
- Missing website URL
- Missing username
- Old passwords if updated_at exists
- Security score

Important:

- Perform sensitive analysis locally after decryption.
- Do not send decrypted passwords to the server.
- Do not log decrypted values.

Prompt 13: Settings

Build settings pages.

Include where schema supports:

- Profile
- Email/account info
- Theme preference
- Default vault/category
- Security settings
- Export warning
- Delete account flow

Do not add unsafe settings.

Prompt 14: Import and Export

Implement import/export.

Import:

- CSV import
- Field mapping
- Validation
- Duplicate detection
- Encrypt before saving

Export:

- Require confirmation
- Warn the user clearly
- Prefer encrypted export
- Never export silently
- Never log exported content

Prompt 15: UI Polish

Polish the app to match the screenshots.

Improve:

- Layout
- Spacing
- Typography
- Buttons
- Inputs
- Modals
- Detail panels
- Sidebar
- Icons
- Empty states
- Hover states
- Focus states
- Dark/light theme if supported
- Mobile responsiveness

The app should feel premium, trustworthy, and simple.

Prompt 16: Security Review

Perform a full security audit.

Check:

- Plaintext secret storage
- Console logs
- Unauthorized data access
- Route protection
- Database policies
- Input validation

- XSS risk
- CSRF risk if applicable
- Clipboard behavior
- Error messages
- Environment variables
- Hardcoded secrets
- Dependency vulnerabilities
- Encryption weaknesses

Fix every critical and high-risk issue.

Produce a security checklist.

Prompt 17: Testing

Add and run tests where practical.

Test:

- Sign up
- Sign in
- Sign out
- Protected routes
- Create password item
- Edit password item
- Delete password item
- Encryption/decryption
- Secure notes
- Vault/category filtering
- Search
- Favorites
- Password generator
- Security health
- Import/export
- Settings
- Responsive layout

Fix failing tests and build errors.

Prompt 18: Production Readiness

Prepare the app for production.

Create or update:

- README
- Setup guide
- Environment variable guide
- Database setup guide
- Security notes
- Known limitations
- Future roadmap

Verify:

- Build passes
- Type checks pass
- No plaintext secrets
- No sensitive logs
- Database access is user-scoped
- UI works on desktop and mobile

Prompt 19: Final Product Review

Review the finished app as a senior security architect and product engineer.

Answer:

1. Is this safe enough for MVP users?
2. What must be fixed before launch?
3. What is missing compared to 1Password?
4. What should not be built yet?
5. What are the top security risks?
6. What are the top product risks?
7. What should be built next?
8. What is the recommended launch scope?

Provide a prioritized action plan.